Griffith UNIVERSITY
Queensland, Australia

2802ICT Intelligent Systems
Trimester 2, 2025

Assignment 1- Programming

- Total marks: 40 (Task 1: 20 marks; Task 2: 20 marks).

Note: Assignment 1-Theoretical (Quiz 1) is worth 10 marks and will be held in the lecture for Week 6.

- Due: Thursday 28 August 2025 at 23:59

- Late submission: Late submission is allowed but a penalty applies. The penalty is defined as the reduction of the mark allocated to the assessment item by 5% of the total weighted mark for the assessment item, for each day that the item is late. Assessment items submitted more than seven days after the due date will be awarded zero marks.

- Extension: You can request for an extension of time on one of two grounds: medical or special circumstances ((e.g. special family or personal circumstances, unavoidable commitments). All requests must be made through the myGriffith portal.

- Individual Work: You must complete this assignment individually and submit an authentic work. This should be your own work. Anyone caught plagiarising will be penalised and reported to the university.

- Presentation: You must present/demonstrate your work in one-on-one conversation with a teaching staff member to discuss all components of this assignment and demonstrate your understanding of the subject. The schedule for the student demonstration will be available before the conversations. Your marks in the assignment will be modified based on this one-on-one conversation. Your work will not be marked if you do not present it.

Objectives:

✓ Formulate a given problem
✓ Demonstrate AI programming skills
✓ Carry out empirical evaluations of different algorithms

Overview:

The purpose of the assignment is to assess your ability to understand and implement various search strategies, to refine them and study their results by running experiments. Please make sure that all code that you submit is your own and is not taken/copied from anywhere else (except for the code provided in the assignment).

The algorithms should be implemented in Python. Your code should be easy to follow.

You will also need to put your findings and analysis in a neatly written up report. Read carefully both tasks before start implementing.

Task 1 - Solving a Maze (20 marks)

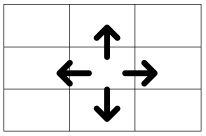
In this task, you are going to solve the following maze search problem by developing an agent in Python using various search strategies.

The maze is described by an NxM board. Each square of the board represents either a free cell or a wall. The goal is to find an optimal path from a starting cell to a destination cell using different search strategies.

You don't have to implement the whole program. Instead, you task is to modify the provided Python program in maze_src.zip.

The rules:

1. Your agent is allowed to move one cell at a time to an adjacent cell (no diagonal moves are

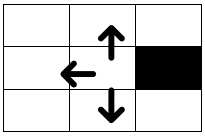


   allowed):

2. If a cell is a wall, the agent cannot move to that cell. For example, in the following image the black square represents a wall. Hence, the agent can only move to the other three cells:

3. All moves cost the same in this task.

The input:



Your code should read an input from a given text file (three sample input files are provided).

The output:

1. The following details should be included in your output (each in a different line):

   a. the solution found by the algorithm, presented as a sequence of moves separated by a dash (-). The four potential moves are L (left), U (up), R (right), and D (down). For example, the sequence R-R-D-D-L-D-D-U represents the path: right, right, down, down, left, down, down, up.

      If no solution is found, you should print "no solution"

   b. the total path cost.

c. the path length (number of moves in the path).

d. number of explored states.

e. the original maze (similar to how it is represented in the text file)

f. a visualisation of the path (if found) - this can be done by simply replacing the empty cell with another character in the original board according to the suggested path. You are free to choose another way to visualise a path, as long as the path and board are easy to understand.

g. any other statistics you want to add - feel free to add here.

Requirements

Software (10 marks):

1. Implement a program that reads a maze from an input file input.txt as described above.
2. For a given maze you need to find a solution, or report of a failure to find one, using the requested algorithms. The algorithms you need to support are:

   (1) Iterative Deepening Search (IDS)

   (2) A*

3. Output your results as described above, for both algorithms.

Report (10 marks):

4. Provide a problem formulation for the maze problem.
5. Provide a description explaining the program's functionality, design decisions, and instructions for use. Include details on any key functions and data structures you have added in the program. For A* you should also explain your heuristic and why have you chosen it.
6. Report all your experiments and results in a neat and clear way.
7. Discuss the results, include any observations or insights you gained from running the tests.
8. Write a conclusion paragraph that summaries and explains your findings.

Suggested experiments:

9. Compare and contrast the different algorithms. For example, compare the solution found, number of explored states, different heuristics when applicable, etc.
10. You should test your code on a variety of test cases, including cases where the goal point is not reachable from the starting point.

You are encouraged to test your program and think of experiments you can perform to assess the effectiveness and efficiency of your algorithms.

Task 2 - Solving Crossword Puzzles (20 marks)

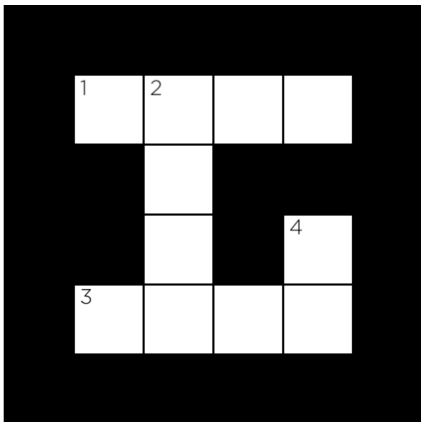
In this task, you are going to write an AI to generate and solve crossword puzzles.

```
$ python generate.py data/structure1.txt data/words1.txt output.png
```

Crossword Puzzles:

How might you go about generating a crossword puzzle? Given the structure of a crossword puzzle (i.e., which squares of the grid are meant to be filled in with a letter), and a list of words to use, the problem becomes one of choosing which words should go in each vertical or horizontal sequence of squares.

How to formulate this problem as a constraint satisfaction problem? Each sequence of squares is one variable, for which we need to decide on its value (which word in the domain of possible words will fill in that sequence). Consider the following crossword puzzle structure.



In this structure, we have four variables, representing the four words we need to fill into this crossword puzzle (each indicated by a number in the above image). Each variable is defined by four values: the row it begins on (its i value), the column it begins on (its j value), the direction of the word (either down or across), and the length of the word. Variable 1, for example, would be a variable represented by a row of 1 (assuming 0 indexed counting from the top), a column of 1 (also assuming 0 indexed counting from the left), a direction of across, and a length of 4.

As with many constraint satisfaction problems, these variables have both unary and binary constraints. The unary constraint on a variable is given by its length. For Variable 1, for instance, the value BYTE would satisfy the unary constraint, but the value BIT would not (it has the wrong number of letters). Any values that don't satisfy a variable's unary constraints can therefore be removed from the variable's domain immediately.

The binary constraints on a variable are given by its overlap with neighbouring variables. Variable 1 has a single neighbour: Variable 2. Variable 2 has two neighbours: Variable 1 and Variable 3. For each pair of neighbouring variables, those variables share an overlap: a single square that is common to them both. We can represent that overlap as the character index in each variable's word that must be the same character. For example, the overlap between Variable 1 and Variable 2 might be represented as the pair (1, 0), meaning that Variable 1's character at index 1 necessarily must be the same as Variable 2's character at index 0 (assuming 0-indexing, again). The overlap between Variable

2 and Variable 3 would therefore be represented as the pair (3, 1): character 3 of Variable 2's value must be the same as character 1 of Variable 3's value.

For this problem, we'll add the additional constraint that all words must be different: the same word should not be repeated multiple times in the puzzle.

The challenge ahead, then, is to write a program to find a satisfying assignment: a different word (from a given vocabulary list) for each variable such that all of the unary and binary constraints are satisfied.

Requirements:

Software (17 marks)

Complete the implementation of `enforce_node_consistency`, `revise`, `ac3`, `assignment_complete`, `consistent`, `order_domain_values`, `selected_unassigned_variable`, and `backtrack` in `generate.py` so that your AI generates complete crossword puzzles if it is possible to do so.

The `enforce_node_consistency` function should update `self.domains` such that each variable is node consistent.

- Recall that node consistency is achieved when, for every variable, each value in its domain is consistent with the variable's unary constraints. In the case of a crossword puzzle, this means making sure that every value in a variable's domain has the same number of letters as the variable's length.
- To remove a value `x` from the domain of a variable `v`, since `self.domains` is a dictionary mapping variables to sets of values, you can call `self.domains[v].remove(x)`.
- No return value is necessary for this function.

The `revise` function should make the variable `x` arc consistent with the variable `y`.

- `x` and `y` will both be `Variable` objects representing variables in the puzzle.
- Recall that `x` is arc consistent with `y` when every value in the domain of `x` has a possible value in the domain of `y` that does not cause a conflict. (A conflict in the context of the crossword puzzle is a square for which two variables disagree on what character value it should take on.)
- To make `x` arc consistent with `y`, you'll want to remove any value from the domain of `x` that does not have a corresponding possible value in the domain of `y`.
- Recall that you can access `self.crossword.overlaps` to get the overlap, if any, between two variables.
- The domain of `y` should be left unmodified.
- The function should return `True` if a revision was made to the domain of `x`; it should return `False` if no revision was made.

The `ac3` function should, using the AC3 algorithm, enforce arc consistency on the problem. Recall that arc consistency is achieved when all the values in each variable's domain satisfy that variable's binary constraints.

- Recall that the AC3 algorithm maintains a queue of arcs to process. This function takes an optional argument called `arcs`, representing an initial list of arcs to process. If `arcs` is `None`, your function should start with an initial queue of all of the arcs in the problem. Otherwise,

your algorithm should begin with an initial queue of only the arcs that are in the list `arcs` (where each arc is a tuple `(x, y)` of a variable `x` and a different variable `y`).
- Recall that to implement AC3, you'll revise each arc in the queue one at a time. Any time you make a change to a domain, though, you may need to add additional arcs to your queue to ensure that other arcs stay consistent.
- You may find it helpful to call on the `revise` function in your implementation of `ac3`.
- If, in the process of enforcing arc consistency, you remove all of the remaining values from a domain, return `False` (this means it's impossible to solve the problem, since there are no more possible values for the variable). Otherwise, return `True`.
- You do not need to worry about enforcing word uniqueness in this function (you'll implement that check in the `consistent` function.)

The `assignment_complete` function should (as the name suggests) check to see if a given `assignment` is complete.

- An `assignment` is a dictionary where the keys are `Variable` objects and the values are strings representing the words those variables will take on.
- An assignment is complete if every crossword variable is assigned to a value (regardless of what that value is).
- The function should return `True` if the assignment is complete and return `False` otherwise.

The `consistent` function should check to see if a given `assignment` is consistent.

- An `assignment` is a dictionary where the keys are `Variable` objects and the values are strings representing the words those variables will take on. Note that the assignment may not be complete: not all variables will necessarily be present in the assignment.
- An assignment is consistent if it satisfies all of the constraints of the problem: that is to say, all values are distinct, every value is the correct length, and there are no conflicts between neighboring variables.
- The function should return `True` if the assignment is consistent and return `False` otherwise.

The `order_domain_values` function should return a list of all of the values in the domain of `var`, ordered according to the least-constraining values heuristic.

- `var` will be a `Variable` object, representing a variable in the puzzle.
- Recall that the least-constraining values heuristic is computed as the number of values ruled out for neighboring unassigned variables. That is to say, if assigning `var` to a particular value results in eliminating `n` possible choices for neighboring variables, you should order your results in ascending order of `n`.
- Note that any variable present in `assignment` already has a value, and therefore shouldn't be counted when computing the number of values ruled out for neighboring unassigned variables.
- For domain values that eliminate the same number of possible choices for neighboring variables, any ordering is acceptable.
- Recall that you can access `self.crossword.overlaps` to get the overlap, if any, between two variables.
- It may be helpful to first implement this function by returning a list of values in any arbitrary order (which should still generate correct crossword puzzles). Once your algorithm is working, you can then go back and ensure that the values are returned in the correct order.
- You may find it helpful to `sort` a list according to a particular `key`: Python contains some helpful functions for achieving this.

The `select_unassigned_variable` function should return a single variable in the crossword puzzle that is not yet assigned by `assignment`, according to the minimum remaining value heuristic and then the degree heuristic.

- An `assignment` is a dictionary where the keys are `Variable` objects and the values are strings representing the words those variables will take on. You may assume that the assignment will not be complete: not all variables will be present in the assignment.
- Your function should return a `Variable` object. You should return the variable with the fewest number of remaining values in its domain. If there is a tie between variables, you should choose among whichever among those variables has the largest degree (has the most neighbors). If there is a tie in both cases, you may choose arbitrarily among tied variables.
- It may be helpful to first implement this function by returning any arbitrary unassigned variable (which should still generate correct crossword puzzles). Once your algorithm is working, you can then go back and ensure that you are returning a variable according to the heuristics.
- You may find it helpful to `sort` a list according to a particular `key`: Python contains some helpful functions for achieving this.

The `backtrack` function should accept a partial assignment `assignment` as input and, using backtracking search, return a complete satisfactory assignment of variables to values if it is possible to do so.

- An `assignment` is a dictionary where the keys are `Variable` objects and the values are strings representing the words those variables will take on. The input assignment may not be complete (not all variables will necessarily have values).
- If it is possible to generate a satisfactory crossword puzzle, your function should return the complete assignment: a dictionary where each variable is a key and the value is the word that the variable should take on. If no satisfying assignment is possible, the function should return `None`.
- If you would like, you may find that your algorithm is more efficient if you interleave search with inference (as by maintaining arc consistency every time you make a new assignment). You are not required to do this, but you are permitted to, so long as your function still produces correct results. (It is for this reason that the `ac3` function allows an `arcs` argument, in case you'd like to start with a different queue of arcs.)

You should not modify anything else in `generate.py` other than the functions the specification calls for you to implement, though you may write additional functions and/or import other Python standard library modules. You may also import `numpy` or `pandas`, if familiar with them, but you should not use any other third-party Python modules. You should not modify anything in `crossword.py`.

Hints:

- For `order_domain_values` and `select_unassigned_variable`, it may be helpful to implement them first without worrying about the heuristics, and then add heuristics later. Your algorithm will still work: it just may end up exploring more assignments than it needs to before finding a solution.
- To run your program, you can run a command like `python generate.py data/structure1.txt data/words1.txt`, specifying a structure file and a words file. If an assignment is possible, you should see the resulting assignment printed. You may also add an additional command-line argument for an image file, as by running `python`

`generate.py data/structure1.txt data/words1.txt output.png`, to generate an image representation of the resulting crossword puzzle as well.

- The `Crossword` class has a `neighbors` function you can use to access all of the neighbors (i.e., overlapping variables) of a particular variable. Feel free to use that any time you need to determine the neighbors of a particular variable!

Short Report (1-2 pages, 3 marks)

- Report all of your experiments and results in a neatly and clear way.
- Discuss the results, include any observations or insights you gained from running the tests.
- Write a conclusion paragraph that summaries and explains your findings.

Submission

a. Your full **source code** (*.py file/s)

b. A copy of your source code in one of the following formats: Microsoft Word (.doc/.docx), Plain text (.txt), or Adobe PDF (.pdf).

c. Your **report** in Microsoft Word (.doc/.docx) or Adobe PDF (.pdf) format.

REMEMBER:
You must need to present and discuss your work in a one-on-one interview with one of the teaching staff

Submission should be done through the link provided in the course website by the due date.